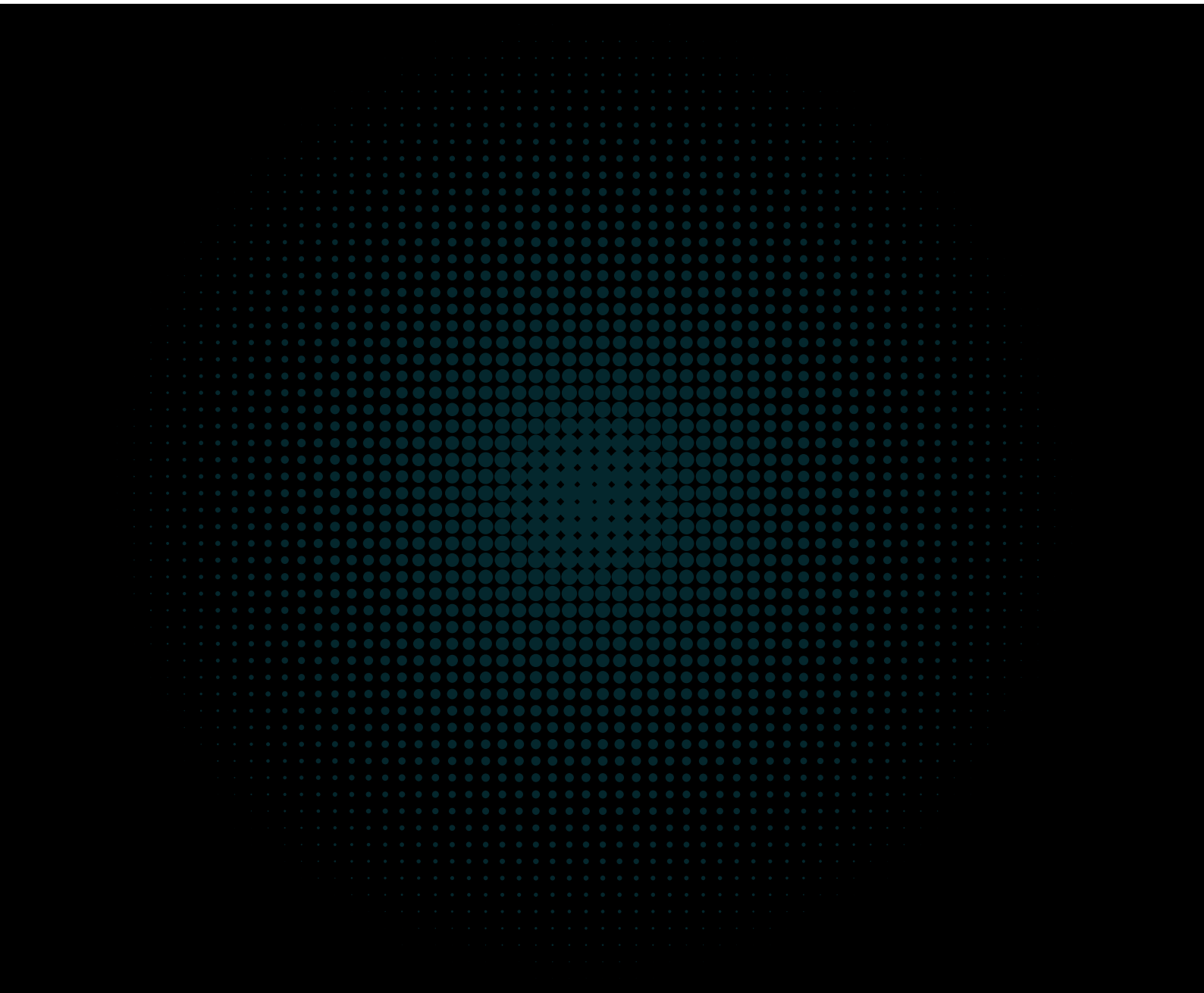
cmta.

CMTAT Equivalency

Functional specifications of the CMTA Token to create CMTAT compliant tokens

Version: v0.3.0
First published: September 2026



CMTAT Equivalency Assessment Criteria

Table of Contents

Document Version

Two distinct versions MUST be distinguished: the version of **this template**, as published by CMTA, and the version of the **filled assessment** produced for the implementation being approved.

Version	Value
Template version — this document, as published by CMTA; pre-filled, MUST NOT be modified by the author of an assessment	`v0.3.0`
Assessment version — the filled document, set by its author	

The template version is not a second value to fill in. An assessment is produced by filling a copy of this document, so the value above **is** the template version that assessment was filled from, and it MUST be carried over unchanged. Only an assessment written in a separate document — a report that reproduces the criteria instead of filling this file — has to quote the template version explicitly.

A filled assessment SHOULD state both numbers on a single line, for example:

> `v0.2.0` (this assessment), filled from CMTA assessment template `v0.3.0`

The two numbers are independent: a filled assessment MAY be revised — for example after a new release of the implementation being approved — without any change to the template, and a new template version MAY be published without the existing assessments being refilled.

Note:

- versions with the `rc` suffix are draft versions.
- version before `1.0` are also draft versions
- both notes above apply to the template version and to the assessment version.
- the template version MUST always be recorded next to the assessment version: criteria IDs are sequential over the whole document and MAY be renumbered from one template version to the next, so an answer given against an earlier template cannot be read against a later one without checking the changelog.

Metadata

This section identifies the implementation being assessed. It MUST be completed before the CMTAT Function Equivalency Table, together with the assessment version in Document Version.

Field	Value
Implementation name	
Target blockchain or distributed ledger	
Implementation language	
Implementation version	
Source repository and commit	
Assessment date	
Assessed by	

The *Implementation version* is the version of the token implementation itself, as returned by criterion 6 when that criterion is supported. It is neither the assessment version nor the template version.

How to Use This Document

- Fill the document in this order: **CMTAT Function Equivalency Table** first, then **Summary**, then **Conclusion**.
- Before starting, fill **Metadata** — what is being assessed — and the assessment version in **Document Version**, next to the template version the assessment is filled from.
- Use the **CMTAT Function Equivalency Table** as the fillable assessment checklist. Each criterion MUST be answered with `y`, `partial`, or `n` in the *Present in implementation being approved* column (see Answer values).
- Use the **Summary** to give the aggregated compliance result of the implementation being approved with the CMTAT standard.
- Use the **Conclusion** to describe, in broad terms, how the implementation being approved works and where it differs from CMTAT.
- Use **Guideline for New Blockchain Implementations** as reference guidance when designing or mapping non-Solidity implementations.

General Note

- The listed functionalities are the **minimal set** required for each module.
- The key words "MUST", "MUST NOT", "REQUIRED", "SHOULD", and "MAY" in this document are to be interpreted as described in RFC 2119 and RFC 8174.

Warning

An implementation MAY satisfy the CMTAT standard while still failing to meet the criteria required for tokenized shares under Swiss law at the underlying-ledger level. In particular, compliance with CMTAT does not, by itself, demonstrate that decentralization-related legal criteria are satisfied.

Summary

> This section MUST be completed **after** the CMTAT Function Equivalency Table. It summarizes the compliance of the implementation being approved with the CMTAT standard.

Scope of the count

The equivalency table contains **61 numbered criteria**:

Category	Count	IDs
Mandatory	19	1–3, 7–11, 14–21, 29–31
Optional	42	4–6, 12–13, 22–28, 32–61

Each criterion MUST be counted exactly once. The non-numbered tables (CMTAT Extended, Implementation Details, Cross-Chain Bridge Support, Restriction, Privacy and Confidentiality) are **not** part of this count; they SHOULD be commented in the Conclusion instead.

Answer values

Each criterion MUST be answered with exactly one of the following values in the *Present in implementation being approved* column:

Value	Meaning
`y`	**Present** — an equivalent feature exists and covers the requirement, even if the name, the signature, or the chain-level mechanism differs from CMTAT Solidity.
`partial`	**Partial** — an equivalent feature exists but covers only a part of the requirement, or covers it with a restriction, a different access control model, or different semantics.
`n`	**Absent** — no equivalent feature is available in the implementation being approved.

Compliance table

Answer	Mandatory (19)	Optional (42)
Present (`y`)		
Partial		
Absent (`n`)		

Each column MUST sum to its total: 19 for mandatory criteria, 42 for optional criteria.

An implementation SHOULD be considered equivalent to CMTAT only if **no mandatory criterion is answered `n`**. Any mandatory criterion answered `partial` MUST be justified in the note below.

Note

> This subsection MUST explain the figures given in the compliance table, and in particular:
>
> - **every `partial` answer**: which part of the requirement is covered, which part is not, and why (chain-level limitation, legal or business choice, different access control model, feature planned for a later version);
> - **every mandatory `n` answer**: why the requirement cannot be met, and what compensating measure (if any) exists;
> - **optional modules left out by design**: when a whole optional module (for example Dividend, Debt, or Snapshot) is answered `n`, it SHOULD be stated once here rather than criterion by criterion.
>
> Example of an entry:
>
> > *Criterion 17 (Deactivate contract) — Partial: the implementation permanently blocks all transfers and mints, but the account itself cannot be deleted from the ledger, since the chain runtime does not allow it. The deactivated state is irreversible and publicly readable.*

▶ Example of a filled compliance table

Answer	Mandatory (19)	Optional (42)

Answer	Mandatory (19)	Optional (42)
Present (`y`)	16	7
Partial	3	4
Absent (`n`)	0	31

CMTAT Function Equivalency Table

Token Attributes

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
1	Name attribute	ERC20 `name`	Public (`view`)				
2	Reference to legally required documentation	`terms`	Public (`view`)				
3	Decimals (no fractions by default)	ERC20 `decimals`	Public (`view`)	- Decimals MUST be set to zero unless governing law permits fractions. - The value MUST be readable, since a holder cannot interpret a balance without it. - CMTAT Solidity allows configurable decimals at deployment			

For CMTAT reference implementations, decimals SHOULD be configurable rather than defaulting to zero, to support use cases beyond tokenized shares in Switzerland.

Note

> This subsection can be used to detail how mandatory token attributes are implemented and to document specific legal, business, or chain-specific cases.

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
4	Ticker symbol attribute	ERC20 `symbol`	Public (`view`)	Optional in the CMTA framework, which lists the attribute as "Ticker symbol (optional)".			
5	Token ID attribute	`tokenId`	Public (`view`)	Optional parameter.			
6	Version attribute	`version()` (`IERC3643Version`, implemented by `VersionModule`)	Public (`view`)	Returns the version of the token implementation, for example `"3.2.0"`. In CMTAT Solidity the value is a constant of the contract code: it changes only through a new deployment or an upgrade, and it is not settable at runtime.			

An official CMTAT implementation SHOULD provide a ticker symbol (criterion 4): every CMTA reference implementation carries one, and a symbol SHOULD be set wherever the token is intended to be held in a wallet or admitted to trading.

For CMTAT reference implementations, `tokenId` and `version` SHOULD both be included.

Note

> This subsection can be used to detail optional token attributes implemented by the target system and to explain specific cases where an optional field is omitted or represented differently.

Token module

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
7	Know total supply	ERC20 `totalSupply`	Public (`view`)				
8	Know balance	ERC20 `balanceOf`	Public (`view`)				
9	Transfer tokens	ERC20 `transfer`	Token holder (`msg.sender`)				
10	Create tokens	`mint` / `batchMint`	Role-restricted (issuer/minter authorized)				

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
11	Cancel tokens	`burn` / `batchBurn` / `burnFrom`	Role-restricted (issuer/burner authorized)	Implementations SHOULD use a dedicated issuer/authorized burn path for forced cancellation scenarios.			

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
12	User-approved cancellation	`burnFrom(address account, uint256 value)`	Role-restricted (`BURNER_FROM_ROLE`) **and** an ERC-20 allowance granted by the token holder	The token holder authorizes the cancellation with an `approve`, and the issuer, or an address it has authorized such as a bridge, performs it. It lets the issuer distinguish a cancellation made to manage supply from one made to carry out a court order. Cancellation by the issuer alone is criterion 11; cancellation by the holder alone is not offered by default — see Self-Burn.			
13	Approve	ERC20 `approve(address spender, uint256 value)`	Token holder	Grants a delegate permission to transfer a specific amount of tokens from the token account. This is optional, but implementations SHOULD include it since secondary market capability may depend on delegated approval to automate trading and settlement for regulated entities. Issuers SHOULD consult relevant trading and settlement venues if listing is contemplated.			

Note

> This subsection can be used to detail how each mandatory function is implemented, including role model, execution flow, and specific chain-level behavior.

Pause module (mandatory)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
14	Pause tokens	`pause`	Role-restricted (pauser/admin authorized)	Pause must prevent all transfers until `unpause` is called.			
15	Unpause tokens	`unpause`	Role-restricted (pauser/admin authorized)				
16	Know pause status	`paused()`	Public (`view`)	Any person MUST be able to determine whether the token is paused; a pause that cannot be read leaves a holder unable to tell why a transfer was refused.			
17	Deactivate contract	`deactivateContract`	Role-restricted (admin authorized)	Must permanently disable the token (except in upgradeability patterns where deactivation behavior is explicitly defined).			
18	Know deactivate status	`deactivated()`	Public (`view`)	Any person MUST be able to determine whether the token has been deactivated. In CMTAT Solidity the function is declared by the draft `IERC8343` interface.			

Note

> This subsection can be used to detail how each mandatory function is implemented, including role model, execution flow, and specific chain-level behavior.

Enforcement

Mandatory

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
19	Freeze	`freeze` or `setAddressFrozen(true)` *(inferred from extracted PDF text)*	Role-restricted (compliance/admin authorized)	Must block transfers to and from a given address. Single-function implementations are acceptable if they set a frozen status.			
20	Unfreeze	`unfreeze` or `setAddressFrozen(false)` *(inferred from extracted PDF text)*	Role-restricted (compliance/admin authorized)	Single-function implementations are acceptable if they clear a frozen status.			
21	Know frozen status	`isFrozen(address account)`	Public (`view`)	Any person MUST be able to determine whether a given address is frozen. On a ledger providing confidentiality the reading MAY be restricted to the issuer, the holder concerned and the third parties the issuer authorizes — see Privacy and Confidentiality.			

Optional

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
22	Enforce a transfer	`forcedTransfer(address from, address to, uint256 value)`	Role-restricted (operator/compliance authorized)	Enforcement transfer is performed via `forcedTransfer`.			
23	Partial freeze	`freezePartialTokens(address account, uint256 value)` / `unfreezePartialTokens(address account, uint256 value)`	Role-restricted (operator/compliance authorized)	Intended only to block a sold amount to avoid double-spend during settlement.			
24	Know active balance	`getActiveBalanceOf(address account)`	Public (`view`)	The balance the holder can still transfer, that is the total balance less the partially frozen amount. Only meaningful where partial freeze (criterion 23) is offered.			

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
25	Know frozen balance	`getFrozenTokens(address account)`	Public (`view`)	The partially frozen amount held on an address. Declared by the draft `IERC7943` interface in CMTAT Solidity. On a ledger providing confidentiality the reading MAY be restricted in the same way as the frozen status (criterion 21).			

Note

> This subsection can be used to detail how each mandatory function is implemented, including role model, execution flow, and specific chain-level behavior.

Transfer restriction (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
26	Conditional transfer request	`RuleConditionalTransferLight.detectTransferRestriction(from, to, value)` / `detectTransferRestrictionFrom(spender, from, to, value)` and `approvedCount(from, to, value)`	Public (`view`)	Request is represented by a transfer restricted until approval count is non-zero.			
27	Conditional transfer approval	`RuleConditionalTransferLight.approveTransfer(from, to, value)` (or `approveAndTransferIfAllowed`)	Role-restricted (compliance/approver authorized)	Approval is consumed on transfer via `transferred(...)`; cancellation via `cancelTransferApproval(...)`.			
28	Assign to whitelist	CMTAT Allowlist: `setAddressAllowlist(account, status)`, `batchSetAddressAllowlist(accounts, status)`, `isAllowlisted(account)`; Rules whitelist: `addAddress`, `removeAddress`, `addAddresses`, `removeAddresses`, `isAddressListed`	Role-restricted for setters; public (`view`) for checks	CMTAT Allowlist and Rules whitelist are alternative whitelist implementations.			

Note

> This subsection can be used to detail the different transfer restrictions available.

Access Control

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
29	Grant role	`grantRole(bytes32 role, address account)` (OpenZeppelin AccessControl via CMTAT/Rules modules)	Role admin (`DEFAULT_ADMIN_ROLE` or role admin)	Used for roles such as `ALLOWLIST_ROLE`, `DEBT_ROLE`, `OPERATOR_ROLE`, `COMPLIANCE_MANAGER_ROLE`.			
30	Revoke role	`revokeRole(bytes32 role, address account)`	Role admin (`DEFAULT_ADMIN_ROLE` or role admin)	AccessControl role removal.			
31	Role attribution	`hasRole(bytes32 role, address account)` / `getRoleAdmin(bytes32 role)`	Public (`view`)	In CMTAT `AccessControlModule`, `DEFAULT_ADMIN_ROLE` is treated as having all roles in `hasRole`.			

Note

> This subsection can be used to detail the concrete authorization model (roles, admins, delegates, approvers) and implementation-specific exceptions. It MAY also be relevant to explain how access control works in the implementation being approved.

Snapshot (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
32	Schedule a snapshot	`scheduleSnapshot(uint256 time)`	Role-restricted (snapshot scheduler/admin authorized)	SnapshotEngine `ISnapshotScheduler`.			
33	Reschedule a snapshot	`rescheduleSnapshot(uint256 oldTime, uint256 newTime)`	Role-restricted (snapshot scheduler/admin authorized)	`newTime` must stay between adjacent scheduled snapshots (not before previous / not after next).			
34	Unschedule a snapshot	`unscheduleLastSnapshot(uint256 time)` / `unscheduleSnapshotNotOptimized(uint256 time)`	Role-restricted (snapshot scheduler/admin authorized)	`unscheduleLastSnapshot` is restricted to the latest scheduled snapshot; `unscheduleSnapshotNotOptimized` supports generic unscheduling.			
35	Snapshot time	`getAllSnapshots()` / `getNextSnapshots()`	Public (`view`)	Returns created snapshot times and pending scheduled times.			
36	Snapshot total supply	`snapshotTotalSupply(uint256 time)`	Public (`view`)	`ISnapshotState`.			
37	Snapshot balance	`snapshotBalanceOf(uint256 time, address tokenHolder)`	Public (`view`)	`ISnapshotState` (see also `snapshotInfo`).			

Note

> This subsection can be used to detail snapshot scheduling and query behavior, including timing constraints and permission specifics.

Dividend (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
38	Distribution create parameters						
39	Distribution set eligibility						
40	Distribution set deposit						
41	Distribution claim deposit						
42	Distribution schedule						
43	Distribution unschedule						

Note

> This subsection can be used to detail dividend/distribution workflow specifics and jurisdiction- or product-specific handling rules. No direct CMTAT Solidity equivalent is currently defined for these items; they are implementation-specific. However, a prototype is available on the CMTA GitHub organization: https://github.com/CMTA/IncomeVault

Credit Events (optional)

ID	Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
44	Flag as default	`setCreditEvents(CreditEvents)` -> `creditEvents().flagDefault`	Role-restricted (issuer/compliance/admin authorized)	Managed in `ICMTATCreditEvents.CreditEvents`.			
45	Remove default flag	`setCreditEvents(CreditEvents)` with `flagDefault = false`	Role-restricted (issuer/compliance/admin authorized)	Same function as ID 44 with a different value.			
46	Flag as redeemed	`setCreditEvents(CreditEvents)` -> `creditEvents().flagRedeemed`	Role-restricted (issuer/compliance/admin authorized)	Managed in `ICMTATCreditEvents.CreditEvents`.			
47	Set rating	`setCreditEvents(CreditEvents)` -> `creditEvents().rating`	Role-restricted (issuer/compliance/admin authorized)	Managed in `ICMTATCreditEvents.CreditEvents`.			

Note

> This subsection can be used to detail how credit event states are updated, governed, and audited in the implementation being approved.

Debt (optional)

ID	Attribute	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
48	Guarantor identifier	`debt().debtIdentifier.guarantor` (set via `setDebt`)	Read: public (`view`); write: role-restricted (`setDebt`)	Debt module (`ICMTATDebt.DebtIdentifier`).			
49	Debtholder representative identifier	`debt().debtIdentifier.debtHolder` (set via `setDebt`)	Read: public (`view`); write: role-restricted (`setDebt`)	Debt module (`ICMTATDebt.DebtIdentifier`).			
50	Unique identifier / hash	`tokenId()` and `terms().doc.documentHash`	Public (`view`)	`tokenId` is optional (implementations MAY omit it); document hash is in `terms` metadata.			
51	Issuance date	`debt().debtInstrument.issuanceDate` (set via `setDebt` / `setDebtInstrument`)	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`ICMTATDebt.DebtInstrument`).			
52	Currency of payments	`debt().debtInstrument.currency` / `debt().debtInstrument.currencyContract`	Read: public (`view`); write: role-restricted (`setDebt*`)	Supports symbol-like string and token/asset contract address.			
53	Par value	`debt().debtInstrument.parValue`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`uint256`).			
54	Minimum denomination	`debt().debtInstrument.minimumDenomination`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`uint256`).			
55	Maturity date	`debt().debtInstrument.maturityDate`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			
56	Interest rate	`debt().debtInstrument.interestRate`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`uint256`).			
57	Coupon payment frequency	`debt().debtInstrument.couponPaymentFrequency`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			

ID	Attribute	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
58	Interest schedule format: A) start date/end date/period; B) start date/end date/day of period; C) date 1/date 2/date 3	`debt().debtInstrument.interestScheduleFormat`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			
59	Interest payment date: A) period; B) specific date	`debt().debtInstrument.interestPaymentDate`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			
60	Day count convention	`debt().debtInstrument.dayCountConvention`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			
61	Business day convention	`debt().debtInstrument.businessDayConvention`	Read: public (`view`); write: role-restricted (`setDebt*`)	Debt module (`string`).			

Note

> This subsection can be used to detail supplementary attributes and to explain specific representation or governance choices made by the implementation being approved.

Guideline for New Blockchain Implementations

If you create a version for another blockchain, use this section to build a correspondence table between the CMTAT framework, the CMTAT Solidity version, and your implementation.

Freeze

> To be compatible with ERC-3643, freeze in CMTAT Solidity is implemented with a single function: `setAddressFrozen(targetAddress, frozenStatus)`. For non-EVM blockchains, implementations MAY separate this into two distinct functions:

```
freeze(address targetAddress)
unfreeze(address targetAddress)
```

Note

> This subsection can be used to detail the choice made by the implementation being approved.

Restriction (optional)

> Transfer restrictions are criteria 26–28 and are optional. The table below is a **reference catalogue** of the restrictions the CMTAT Solidity stack can apply, taken from the Rules repository; it is not part of the equivalency count. An implementation MAY offer any subset of these restrictions, none of them, or restrictions that have no CMTAT Solidity equivalent.

In CMTAT Solidity a restriction is a **rule**: a contract that answers whether a movement of value is allowed.

- A rule is plugged directly into the token, or several rules are composed behind a **RuleEngine**, which returns the first non-zero restriction code — so the order of the rules decides which code a rejection reports.
- Each rule exposes a **read path** — `detectTransferRestriction` / `canTransfer` and their `…From` variants — which MUST NOT revert and returns an ERC-1404 restriction code, `0` meaning no restriction.
- Each rule also exposes a **write path** — `transferred`, `created`, `destroyed` — called by the token once it has decided to move the value. It reverts to block the operation and MAY update the rule state.

Mint and burn use the same path with one party missing — a mint is a movement from the zero address, a burn a movement to the zero address — so a rule does not necessarily apply to them. The table states, for each restriction, what it checks on a transfer, on a mint, and on a burn.

Restriction	CMTAT Solidity rule	On transfer	On mint	On burn	Notes	Present in implementation being approved (`y/partial/n`)	Implementation details
Whitelist	`RuleWhitelist`	Sender and receiver MUST be listed; the spender of a `transferFrom` optionally	Not checked	Not checked			
Aggregated whitelists	`RuleWhitelistWrapper`	Sender and receiver MUST be listed in at least one of the aggregated lists	Not checked	Not checked	An empty wrapper rejects every transfer (fail-closed).		
Receiver whitelist	`RuleReceiverWhitelist`	Receiver only	Receiver MUST be listed	Not checked	Reproduces ERC-3643 eligibility: screening only the receiver lets a de-listed holder still exit a position.		
Spender whitelist	`RuleSpenderWhitelist`	Spender of a `transferFrom` MUST be listed; sender and receiver are not checked	Not checked	Not checked			
Blacklist	`RuleBlacklist`	Blocks a listed sender, receiver or spender	Blocks a listed minter	Blocks a listed burner			
Sanctions list	`RuleSanctionsList`	Blocks a sanctioned sender, receiver or spender	Blocks a sanctioned minter	Blocks a sanctioned burner	Reads an external sanctions oracle. An unset oracle allows everything (fail-open).		
Whitelist and frozen list (ERC-2980)	`RuleERC2980`	Receiver MUST be whitelisted; sender, receiver and spender MUST NOT be frozen	Blocks a frozen minter	Blocks a frozen burner			

Restriction	CMTAT Solidity rule	On transfer	On mint	On burn	Notes	Present in implementation being approved (`y/partial/n`)	Implementation details
Identity registry	`RuleIdentityRegistry`	Receiver MUST be verified; sender and spender are opt-in checks	Not checked	Not checked	Calls `isVerified` on an ERC-3643 identity registry. An unset registry allows everything (fail-open).		
Maximum total supply	`RuleMaxTotalSupply`, `RuleMaxTotalSupplyERC3643`	Not checked	Rejects a mint that would take the total supply over the cap	Not checked	The `…ERC3643` variant exists because an ERC-3643 token calls compliance after moving the value.		
Reserve-backed supply cap	`RuleChainlinkPoR`, `RuleChainlinkPoRERC3643`	Not checked	Rejects a mint that would take the total supply over the reserves reported by a proof-of-reserve feed	Not checked	A missing, broken or stale feed rejects every mint (fail-closed).		
Maximum balance per address	`RuleMaxBalance`	Receiver's balance after the transfer MUST stay under the cap	Receiver's balance after the mint MUST stay under the cap	Not checked	The cap counts tokens per address, so splitting a position across wallets defeats it unless one address per investor is enforced.		
Conditional transfer	`RuleConditionalTransferLight`, `RuleConditionalTransferLightMultiToken`	The exact transfer MUST have been approved beforehand; the approval is consumed	Not checked	Not checked	Criteria 20–21.		
Per-minter quota	`RuleMintAllowance`	Not checked	Debits the minter's quota and rejects the mint once it is exhausted	Not checked	Needs the token to forward the spender on mint (CMTAT `v3.3` or later).		

Note

> This subsection can be used to detail which restrictions the implementation being approved applies, where the logic lives (in the token, in an external module, or in the chain runtime), in which order the restrictions are evaluated, and what a rejected operation returns to the caller — a revert, an error code, or a status returned by a read-only entry point.
>
> Restrictions that have no CMTAT Solidity equivalent SHOULD be listed here as well, together with the behaviour of each restriction when its external data source (oracle, registry, list) is unset or unavailable: rejecting every operation (fail-closed) and allowing every operation (fail-open) are both defensible, but the choice MUST be explicit.

Version

> CMTAT Solidity exposes the version of the implementation through the ERC-3643 function `version()`, provided by the `VersionModule`. The returned value is a compile-time constant (for example `"3.2.0"`) following semantic versioning. It identifies the version of the token implementation; it is neither the version of the issued security nor the version of this assessment document.
>
> Versioning is an optional feature (criterion 6). Implementations on other blockchains MAY expose it differently:
>
> - as a constant returned by a read-only entry point, as in CMTAT Solidity;
> - through the chain-native contract or package metadata, when the target chain already versions the deployed code;
> - as a state variable restricted to an administrator role. In that case, the implementation MUST ensure that the value cannot be desynchronized from the deployed code, typically by writing it only in the initialization or upgrade path.
>
> For an upgradeable implementation, the version SHOULD be updated by the upgrade itself, so that an off-chain observer can always determine which code is live.

Note

> This subsection can be used to detail the choice made by the implementation being approved.

CMTAT Extended

In the table below, the CMTAT framework extended features are mapped to Solidity features.

CMTAT Functionalities	CMTAT Solidity corresponding features	CMTAT Allowlist	CMTAT Light	CMTAT Debt	CMTAT Standard	Present in implementation being approved (`y/partial/n`)	Implementation details
On-chain snapshot	`snapshotModule` and `snapshotEngine`	✔	✘	✔	✔		
Forced transfer	`forcedTransfer`	✔	✘	✔	✔		
Forced burn	`forcedBurn`	✘	✔	✘	✘		
Freeze partial token	`freezePartialTokens` / `unfreezePartialTokens`	✔	✘	✔	✔		
Integrated whitelisting/allowlisting	CMTAT Allowlist	✔	✘	✘	✘		
External whitelisting/allowlisting	CMTAT with rule whitelist	✘	✘	✔	✔		
RuleEngine / transfer hook	CMTAT with RuleEngine	✘	✘	✔	✔		
Upgradeability	CMTAT Upgradeable version	✔	✔	✔	✔		
Fee payer / gasless	CMTAT with ERC-2771 module	✔	✘	✘	✔		

Note

> This section can be used to detail supplementary features implemented beyond the mandatory baseline and specific cases in the target chain. For non-EVM blockchains, it MAY be relevant to explain how gasless/gas sponsorship and upgradeability work in the particular blockchain targeted.

Forced Burn and Forced Transfer

> In the standard burn function, tokens from a frozen wallet MUST NOT be burnable. CMTAT offers `forcedTransfer` to force a transfer or a burn.
>
> If `forcedTransfer` is not available, implementations MAY implement only `forcedBurn` (as in CMTAT Light). Implementations MAY also implement both. In that case, only `forcedBurn` SHOULD burn tokens, and `forcedTransfer` SHOULD NOT burn tokens.
>
> With the CMTAT Solidity version, when `forcedTransfer` is available, `forcedBurn` is not implemented to reduce contract code size. This limitation MAY not apply to other blockchains.

Note

> This subsection can be used to detail the choice made by the implementation being approved.

Implementation Details

Functionalities	CMTAT Solidity	Access Control (CMTAT Solidity)	Note	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
Mint while pause	✔	Role-restricted (minter/issuer authorized)	Dedicated cross-chain mint (for example `crosschainMint`) cannot be performed while paused.			
Burn while pause	✔	Role-restricted (burner/issuer authorized)	Dedicated cross-chain burn (for example `crosschainBurn`) cannot be performed while paused.			
Self-Burn for everyone	✘	Not permitted	Token holders cannot burn their own tokens; only authorized addresses can burn.			
Self-Burn for authorized addresses	✔	Role-restricted (authorized burner)				
Standard burn on a frozen address	✘	Not permitted in standard burn path	Requires `forcedTransfer` or `forcedBurn`.			
Burn tokens with `forcedTransfer`	✔	Role-restricted (operator/compliance authorized)	See notes above.			

Note

> This subsection can be used to detail how the implementation being approved behaves in each case above, including the role model and the specific chain-level behavior while the token is paused or an address is frozen.

Self-Burn

> Only the issuer and authorized addresses (not the token holder) can burn a token in CMTAT Solidity, which reflects legal requirements in several jurisdictions.
>
> The CMTA framework permits it: functionality 41, *user-approved cancel*, states that the functionality "also allows token holders to cancel their own tokens". An implementation MAY therefore offer self-burn where its legal or business context allows. The holder-authorized cancellation that the issuer performs is criterion 12; what is described here is the cancellation the holder performs alone.
>
> An implementation that does offer self-burn SHOULD state so here, together with the legal basis on which it is offered, so that an assessment records which of the two arrangements was adopted rather than leaving it to be inferred.

Cross-Chain Bridge Support

> **This feature is NOT part of the CMTAT specification directly.** Cross-chain transferability is not a requirement of the CMTAT standard and is not part of the equivalency criteria above. It is an **optional module offered by the CMTAT Solidity implementation**, documented here only as a reference so that new implementations MAY map equivalent functionality if they choose to support bridging. An implementation MAY be fully CMTAT-equivalent without providing any cross-chain capability.

CMTAT Solidity supports cross-chain bridging through a **burn-and-mint** model rather than lock-and-mint: tokens are burned on the source chain by an authorized bridge and minted on the destination chain by an authorized bridge. Two complementary standards are implemented in the optional cross-chain module:

- **ERC-7802** — a minimal, bridge-agnostic interface for cross-chain mint and burn. This is the primitive that any compliant token bridge can call.
- **Chainlink CCIP (Cross-Chain Token / CCT standard)** — administrative hooks (`CCIPModule`) that let the token register with the CCIP token admin registry.

A third arrangement sits **outside** the token rather than in it: **CMTAT-LayerZero** is an adapter built on the LayerZero V2 OFT standard, which holds the bridge authorization itself and calls the token to burn on the source chain and mint on the destination chain. It ships in two forms:

- `LayerZeroAdapterERC7802` — calls the ERC-7802 entry points described above, and is the form CMTA recommends where the token implements ERC-7802;
- `LayerZeroAdapter` — calls the ERC-3643 `mint` and `burn` where the token does not, which is the reuse case described below.

Each adapter carries its own pause, controlled by the adapter owner and independent of the token's. An implementation MAY support several bridges at once; each one then holds its own authorization, so that one can be revoked without interrupting the others.

The cross-chain mint/burn entry points are **not** the standard `mint` / `burn` functions: they are dedicated functions restricted to the trusted bridge via a specific role, and they are blocked while the contract is paused (consistent with the *Mint while pause* / *Burn while pause* rows in Implementation Details).

In CMTAT Solidity, the standard `mint` and `burn` remain available while the contract is paused: the pause check is carried by the authorization hook of each entry point (`_checkTokenBridge` is `whenNotPaused`, `_authorizeMint` is not), not by the shared mint/burn path.

An implementation that does **not** provide dedicated cross-chain entry points, and instead reuses the standard `mint` and `burn` functions for bridge operations:

- SHOULD apply the pause check to those functions on the bridge path, so that a cross-chain movement is subject to the same checks as a standard transfer;
- otherwise lets tokens keep moving across chains while transfers are frozen on each of them — a bridge burn on the source chain followed by a bridge mint on the destination chain is economically a transfer between two chains;
- SHOULD state which of the other transfer checks (freeze, partial freeze, allowlist, rule engine or transfer hook) the bridge path enforces, and why any of them is skipped.

Requirement	CMTAT Solidity corresponding feature	Access Control (CMTAT Solidity)	Notes	Present in implementation being approved (`y/partial/n`)	Access Control (implementation being approved)	Implementation details
Cross-chain mint (ERC-7802)	`crosschainMint(address to, uint256 value)`	Role-restricted (`CROSS_CHAIN_ROLE`, trusted token bridge); blocked while paused	Authenticates the bridge with `msg.sender` (not `_msgSender()`) so a relayer/forwarder cannot impersonate the bridge. Emits `CrosschainMint`.			
Cross-chain burn (ERC-7802)	`crosschainBurn(address from, uint256 value)`	Role-restricted (`CROSS_CHAIN_ROLE`, trusted token bridge); blocked while paused	Does **not** require an ERC-20 allowance from `from`, following the Optimism Superchain ERC-20 and OpenZeppelin `ERC20Bridgeable` design. Emits `CrosschainBurn`.			
Advertise ERC-7802 support	`supportsInterface(type(IERC7802).interfaceId)`	Public (`view`)	ERC-165 discovery so bridges can detect ERC-7802 compatibility.			
Set CCIP admin	`setCCIPAdmin(address newAdmin)`	Role-restricted (`DEFAULT_ADMIN_ROLE`)	Chainlink CCIP (CCT) integration. The CCIP admin only registers the token with the CCIP token admin registry and has no other powers; 1-step transfer, `address(0)` revokes.			
Get CCIP admin	`getCCIPAdmin()`	Public (`view`)	Returns the current CCIP admin.			
Bridge burn against an allowance	`burnFrom(address account, uint256 value)`	Role-restricted (`BURNER_FROM_ROLE`); blocked while paused; **and** an ERC-20 allowance granted by `account`	Declared by the same `ERC20CrossChainModule` as the ERC-7802 entry points, and usable by a Chainlink CCIP token pool — the interface notes it carries no `data` parameter for that reason. Spends the allowance and emits `Spend` alongside `BurnFrom`. Same function as criterion 12; see the note below on when to prefer it to `crosschainBurn`.			

Note

> - The trusted bridge holds `CROSS_CHAIN_ROLE`. A bridge MAY `renounceRole` to drop its privileges; this only deprives it of cross-chain mint/burn and has no other effect, but such a bridge should then be considered compromised and not reused.
> - `burnFrom` and `crosschainBurn` bound the bridge's authority differently, and the choice between them is a trust decision rather than a matter of style. `crosschainBurn` deliberately requires no allowance, so a bridge holding `CROSS_CHAIN_ROLE` can cancel the tokens of **any** address; `burnFrom` spends an allowance, so the amount a bridge can cancel is capped per holder by what that holder has approved, and a compromised bridge cannot reach a holder who has approved nothing. The cost is that the holder MUST approve first, which adds a transaction to the bridge flow and does not suit a design in which the bridge burns without the holder acting.
> - CMTAT Solidity also exposes self-`burn` (guarded by `BURNER_SELF_ROLE`) alongside bridging, likewise role-restricted and blocked while paused.
> - This subsection can be used to detail whether and how the implementation being approved supports bridging, which standard(s) or bridge(s) it targets, and the trust/role model applied to the bridge on the target chain. For non-EVM blockchains, ERC-7802 and CCIP may not be directly applicable; an equivalent burn-and-mint bridge model MAY be documented instead.

Privacy and Confidentiality

> **This section is NOT part of the CMTAT specification and is NOT an equivalency criterion.** CMTAT Solidity targets public EVM chains, where all token state is readable by anyone. This section exists so that an implementation targeting a blockchain or a distributed ledger offering some level of privacy or confidentiality can document what stays public, what is hidden, and who is still able to read it.
>
> On a public EVM chain, every element of the table below is public: contract storage is readable by any node even when a variable is declared `private` in Solidity, and every transfer, mint, burn, freeze, and allowlist update is visible in the transaction data and in the emitted events. An implementation on a privacy-enabled ledger MAY hide part of this state. It MUST then document how the hidden state is protected, and who can still read it — in particular whether the issuer retains the visibility required by the CMTAT features it claims (snapshot, dividend, forced transfer, freeze).

Visibility values

Value	Meaning
`public`	Readable by anyone with access to the ledger, as in CMTAT Solidity.
`private`	Not readable from the ledger; only the holder of a specific right (private key, view key, role, node, disclosure credential) can read it.
`partial`	Partly hidden — for example the amount is encrypted but the participants are public, the value is visible only to the parties of the transaction, or it is disclosed only in aggregated or delayed form.

Privacy table

Data	Visibility in CMTAT Solidity	Visibility in the implementation being approved (`public/private/partial`)	Available to the issuer (`y/n`)	Other readers (role or address)	Implementation details
Balance of an address	`public` — `balanceOf` is a public view function and the balance mapping is readable from storage				
Transfer amount	`public` — carried in the transaction calldata and in the `Transfer` event				
Transfer participants (sender and recipient)	`public` — indexed `from` and `to` in the `Transfer` event				
Total supply	`public` — `totalSupply` is a public view function; mint and burn are publicly traceable				
Token decimals	`public` — `decimals` is a public view function				
Frozen / blacklisted addresses	`public` — `isFrozen` / `getFrozenTokens` are public view functions and freeze operations emit events				
Allowlisted / whitelisted addresses (if applicable)	`public` — `isAllowlisted` (CMTAT Allowlist) and `isAddressListed` (Rules whitelist) are public view functions				

Readers

> Typical readers to consider when filling the *Other readers* column. An implementation SHOULD list every case that applies, not only the most restrictive one:
>
> - **everyone** — the value is public on the ledger;
> - **the account holder** — an address can read its own balance and its own transfers, but not those of other holders;

> - **the counterparty of a transfer** — sender and recipient both see the amount, third parties do not;
> - **the issuer or an administrator role** — able to read all balances, typically because the corporate actions covered by this document (snapshot, dividend, forced transfer, freeze) require it;
> - **an auditor, a regulator, or a court-appointed third party** — granted a read credential (view key, decryption share, observer node) permanently or on request;
> - **the operator of the ledger or the validator nodes** — on a permissioned ledger the nodes hosting the data may see it even when the public cannot;
> - **nobody** — the value is recoverable only by the key holder, and is lost with the key.

Note

> This subsection is here to provide additional notes regarding privacy and confidentiality. It SHOULD describe **how privacy and confidentiality work on the particular blockchain targeted**, for example: encrypted balances under homomorphic encryption, a shielded pool with zero-knowledge proofs and view keys, confidential amounts with range proofs, per-participant data segregation on a permissioned ledger, or off-chain balances anchored on-chain by a commitment.
>
> It SHOULD also state the consequences for the CMTAT features: how the total supply can be audited when individual balances are hidden, how a snapshot or a dividend is computed on hidden balances, how the freeze and allowlist checks are enforced without revealing the lists, and what an issuer, an auditor, or a regulator has to do to obtain a disclosure.
>
> Privacy MUST NOT remove a mandatory capability: if the issuer cannot read a balance, the implementation MUST still explain how it performs the operations the mandatory criteria require on that balance.

Supplementary features

> This section MAY be used to document supplementary features beyond the CMTAT standard that are present in the implementation being approved.

Conclusion

> This section MUST describe, in broad terms, **how the implementation being approved works technically**, so that a reader who has not gone through the tables can understand the design and its main differences with the CMTAT specification. It SHOULD cover at least the following points:
>
> - **Token model**: the underlying token primitive of the target blockchain (for example ERC-20, SPL token, Soroban token interface, UTXO-based asset), and how balances, total supply, and decimals are represented.
> - **Architecture**: single contract or several modules/programs, how they are linked (inheritance, composition, external calls, on-chain registry), and the upgradeability strategy (proxy, native chain upgrade, immutable with redeployment).
> - **Access control model**: which roles exist, who holds the administrator role, how roles are granted and revoked, and how these roles map to the CMTAT roles.
> - **Transfer control flow**: which checks are applied on a transfer (pause, freeze, partial freeze, allowlist, rule engine or transfer hook), in which order, and where this logic lives (inside the token, in an external module, or in the chain runtime).

> - **Issuance and cancellation**: the mint and burn paths, forced transfer and forced burn, and the behaviour on a frozen address or while the contract is paused.
> - **Data and metadata storage**: how terms, `tokenId`, debt attributes, and credit events are stored (on-chain state, hash with off-chain document, or chain-native metadata).
> - **Main differences with the CMTAT Solidity implementation**, and the reason for each one (chain constraints, legal context, performance, code size).
> - **Known limitations** and features that are planned but not yet implemented.
>
> The Summary gives the counts; this section gives the technical explanation behind them and MUST stay consistent with it.

Reference

Submodules used in this project and current checked-out versions:

Submodule	Repository	Version	Commit
CMTAT	https://github.com/CMTA/CMTAT	`v3.3.0-rc3`	`658672f190d56d3f61663a7d6d51962b8980df70`
SnapshotEngine	https://github.com/CMTA/SnapshotEngine	`v0.5.0`	`aa089353605cd1b0e555d22b62aa4fbeaae7df25`
RuleEngine	https://github.com/CMTA/RuleEngine	`v3.0.0-rc6`	`ca75429c581a2eb9043e4719561e941d0b2e1206`
Rules	https://github.com/CMTA/Rules	`v0.6.0`	`283efe723225c89729fd618852a9c2705a47180b`
CMTAT-Confidential	https://github.com/CMTA/CMTAT-Confidential	`v1.0.0`	`285ed93721dfbbc147932bd450aa057126e70e84`
CMTAT-LayerZero	https://github.com/CMTA/CMTAT-LayerZero	`v0.2.0` + 1 commit	`e57ca4f076e44ed5a08fdfce9379e2a82925d7cf`
private-CMTAT-aztec	https://github.com/taurushq-io/private-CMTAT-aztec	`0.1.1` + 25 commits	`61f4220d5565840fd4fcdd2b723c9f55eb824c60`

The first four are the implementations the criteria are mapped against. The last three are referenced by the Cross-Chain Bridge Support and Privacy and Confidentiality sections, which are outside the equivalency count.